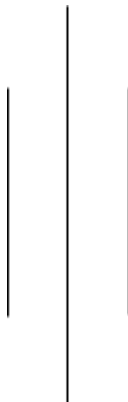


# PRACTICAL FILE OF NUMERICAL METHODS



For partial fulfillment of final examination of  
Numerical Methods [MAT 303]

**Submitted by:**

Janak Singh Dhimi

Roll No.:- 079/530



**Submitted to:**

Department of Mathematics

Tri-Chandra Multiple Campus, Tribhuvan University

Ghantaghar, Kathmandu

**DATE:** 2082-08-23

## Program 1. Bisection Method

### Output:

```
Command Window
Computed on: 2025-12-05 11:47:46
Enter lower guess a: 0
Enter upper guess b: 1
Enter error tolerance: 0.001
Required iterations = -1

Iter      a          b          c          f(c)
1         0.000000    1.000000    0.500000   -0.053222
2         0.500000    1.000000    0.750000    0.856061
3         0.500000    0.750000    0.625000    0.356691
4         0.500000    0.625000    0.562500    0.141294
5         0.500000    0.562500    0.531250    0.041512
6         0.500000    0.531250    0.515625   -0.006475
7         0.515625    0.531250    0.523438    0.017362
8         0.515625    0.523438    0.519531    0.005404
9         0.515625    0.519531    0.517578   -0.000545
10        0.517578    0.519531    0.518555    0.002427

Root = 0.518555
Converged in 10 iterations
>> |
```

## Program 2. Newton-Raphson Method

### Output:

```
Command Window
Computed on: 2025-12-05 11:59:40
Enter initial guess: 0.5
Enter error tolerance: 0.0001

Count      a          f(a)
0          0.500000    -0.053222
1          0.518026     0.000817

Root = 0.517757
Converged in 2 iterations
>> |
```

### Program 3. Secant Method

#### Output:

```
Command Window
Computed on: 2025-12-05 12:09:41
Enter first point a: 5
Enter second point b: 6
Enter error tolerance: .0001

Count      a          b          c          f(c)
0          5.000000    6.000000    3.790123    36.870347
1          6.000000    3.790123    3.216852    19.157132
2          3.790123    3.216852    2.596849    6.365398
3          3.216852    2.596849    2.288324    2.034542
4          2.596849    2.288324    2.143386    0.396221
5          2.288324    2.143386    2.108333    0.034953
6          2.143386    2.108333    2.104942    0.000698
7          2.108333    2.104942    2.104873    0.000001

Root = 2.104873
Converged in 8 iterations
>> |
```

## Program 4. False Position Method

### Output:

```
Command Window
Computed on: 2025-12-05 15:34:10
Enter first point a: 1
Enter second point b: 3
Enter error tolerance: 0.0001

Count      a          b          c          f(c)
0          1.600000    3.000000    1.600000    -3.864000
1          1.902821    3.000000    1.902821    -1.828307
2          2.029555    3.000000    2.029555    -0.729612
3          2.077625    3.000000    2.077625    -0.270782
4          2.095126    3.000000    2.095126    -0.097757
5          2.101401    3.000000    2.101401    -0.034938
6          2.103638    3.000000    2.103638    -0.012442
7          2.104434    3.000000    2.104434    -0.004425
8          2.104717    3.000000    2.104717    -0.001573
9          2.104817    3.000000    2.104817    -0.000559
10         2.104853    3.000000    2.104853    -0.000199
11         2.104866    3.000000    2.104866    -0.000071

Root = 2.104866
Converged in 12 iterations
>> |
```

## Program 5. Fixed Point Iteration Method

### Output:

```
Command Window
Computed on: 2025-12-05 15:59:50
Enter initial guess: 0.5
Enter error tolerance: 0.0001
Enter maximum iterations: 50

Count      x          f(x) = g(x)
0          0.500000    0.877583
1          0.877583    0.639012
2          0.639012    0.802685
3          0.802685    0.694778
4          0.694778    0.768196
5          0.768196    0.719165
6          0.719165    0.752356
7          0.752356    0.730081
8          0.730081    0.745120
9          0.745120    0.735006
10         0.735006    0.741827
11         0.741827    0.737236
12         0.737236    0.740330
13         0.740330    0.738246
14         0.738246    0.739650
15         0.739650    0.738705
16         0.738705    0.739341
17         0.739341    0.738912
18         0.738912    0.739201
19         0.739201    0.739007
20         0.739007    0.739138
21         0.739138    0.739050

Root is: 0.739138
Converged in 21 iterations
>> |
```

## Program 6. Gauss Elimination Method

### Output:

```
Command Window
Computed on: 2025-12-05 16:27:02
Enter the coefficient matrix A (n x n):
Matrix A: [2 -1 1; 1 3 2; 1 -1 2]
Vector b (n x 1): [3; 12; 0]
The solution vector x is:
    3
    3
    0
>> |
```

## Program 7. Gauss Jordan Method

### Output:

```
Command Window
Computed on: 2025-12-05 22:41:21
Enter the coefficient matrix A (n x n):
Matrix A: [2 -1 1; 1 3 2; 6 -1 5]
Vector b (n x 1): [3; 6; 0]
The solution vector x is:
    9.3750
    5.6250
   -10.1250
>> |
```

## Program 8. Gauss Seidel Method

### Output:

```
Command Window
Computed on: 2025-12-05 22:53:22
Matrix A (n x n matrix): [10 2 1; 2 20 -2; -2 3 10]
Vector b (n x 1 vector): [9; -44; 22]
Enter error tolerance: 0.0001
Enter maximum iterations: 100
Iteration 1:
    0.9000
   -2.2900
    3.0670
Iteration 2:
    1.0513
   -1.9984
    3.0098
Iteration 3:
    0.9987
   -1.9989
    2.9994
Iteration 4:
    0.9998
   -2.0000
    3.0000
Iteration 5:
    1.0000
   -2.0000
    3.0000
Iteration 6:
    1.0000
   -2.0000
    3.0000
Solution vector x:
    1.0000
   -2.0000
    3.0000
Converged in 5 iterations
>> |
```

## Program 9. Gauss Jacobi Method

### Output:

```
Command Window
Enter the matrix A (n x n matrix):
Matrix A (n x n matrix): [10 1 1; 2 10 1; 2 2 10];
Vector b (n x 1 vector): [12; 13; 14];
Enter the error tolerance: 0.001;
Enter the maximum number of iterations: 20;
Iteration 1:
    1.2000
    1.3000
    1.4000
Iteration 2:
    0.9300
    0.9200
    0.9000
Iteration 3:
    1.0180
    1.0240
    1.0300
Iteration 4:
    0.9946
    0.9934
    0.9916
Iteration 5:
    1.0015
    1.0019
    1.0024
Iteration 6:
    0.9996
    0.9995
    0.9993
Iteration 7:
    1.0001
    1.0002
    1.0002
The solution vector x is:
    1.0001
    1.0002
    1.0002
Converged in 6 iterations
>> |
```

## Program 10. Matrix Inversion Method

Output:

```
Command Window
Computed on: 2025-12-06 14:17:30
Enter the matrix A (n x n matrix):
Matrix A (n x n matrix): [2 -1 1; 1 3 2; 1 -1 2]
Vector b (n x 1 vector): [3; 12; 0]
The solution vector x is:
    3.0000e+00
    3.0000e+00
   -5.5511e-17
>> |
```

## Program 11. Runga-Kutta Method

### Output:

```
Command Window
Computed on: 2025-12-06 14:38:55
Enter initial value of x (x0): 0
Enter step size h: 0.1
Enter the final value of x (xn): 0.3
Enter initial value of y (y0): 1

Iteration      x      Y      k1      k2      k3      k4
1      0.0000  1.0000  1.0000  0.9955  0.9955  0.9836
2      0.1000  1.0994  0.9836  0.9665  0.9664  0.9456
3      0.2000  1.1960  0.9456  0.9223  0.9221  0.8971

The solution at x = 0.3000 is: 1.2882
>> |
```

## Program 12. Trapezoidal Method

Output:

```
Command Window
Computed on: 2025-12-06 14:44:49
Enter lower limit of integral: 0
Enter upper limit of integral: 1
Enter number of sub-intervals: 4
The value of the integral is:
I = 0.782794
>> |
```

### Program 13. Simpson's $\frac{1}{3}$ Method

Output:

```
Command Window
Computed on: 2025-12-06 16:30:54
Enter the lower limit of integration (a): 0
Enter the upper limit of integration (b): 1
Enter the number of subintervals (n, must be even): 4
The integral of the function from 0.000000 to 1.000000 is approximately:
I = 0.785392
>> |
```

## Program 14. Simpson's $\frac{3}{8}$ Method

Output:

```
Command Window
Computed on: 2025-12-06 16:47:42
Enter the lower limit of integration (a): 0
Enter the upper limit of integration (b): 1
Enter the number of subintervals (n, must be even): 6
The integral of the function from 0.000000 to 1.000000 is approximately:
I = 0.785398
>> |
```

## Program 15. Newton Forward Difference Method

### Output:

```
Command Window
Computed on: 2025-12-06 17:28:01
Enter the x values (data points): [1 2 3 4]
Enter the corresponding y values (data points): [1 8 27 64]
Forward Difference Table:
    1    7   12    6
    8   19   18    0
   27   37    0    0
   64    0    0    0
Enter the value of X to evaluate the polynomial: 2.5
The value of the polynomial at X = 2.5000 is: 15.6250
>> |
```

## Program 16. Newton Backward Difference Method

Output:

```
Command Window
Computed on: 2025-12-06 17:49:08
Enter the x values (data points): [10 20 30 40]
Enter the corresponding y values (data points): [100 400 900 1600]
Backward Difference Table:
    100      0      0      0
    400    300      0      0
    900    500    200      0
   1600    700    200      0
Enter the value of X to evaluate the polynomial: 35
The value of the polynomial at X = 35.0000 is: 1225.0000
>> |
```

## Program 17. Lagrange Interpolation Method

### Output:

```
Command Window
Computed on: 2025-12-06 18:27:13
Enter the x values (data points): [1 2 3]
Enter the corresponding y values (data points): [1 4 9]
Enter the value of X to evaluate the polynomial: 2.5
The value of the polynomial at X = 2.5000 is: 6.2500
>> |
```

### Program 18. (TU 2080)

Write a Matlab program to find a real root of a non-linear equation  $f(x) = 0$  using Newton - Raphson algorithm.

#### Output:

```
Command Window
Computed on: 2025-12-06 21:15:04
Enter the initial guess: 1
Enter the error tolerance: 0.0001
Enter maximum number of iterations: 20

Iter      x           f(x)
1         1.500000     0.875000
2         1.347826     0.100682
3         1.325200     0.002058
4         1.324718     0.000001
5         1.324718     0.000000

The approximate root is: 1.324718
Converged in 5 iterations
>> |
```

### Program 18. (TU 2081)

Write a Matlab program to solve the IVP:  $y' = f(x,y)$ ,  $y(x_0) = y_0$  at  $x = x_p$  RK-2 method with step size  $h$ .

#### Output:

```
Command Window
Computed on: 2025-12-06 19:55:19
Enter initial value x0: 0
Enter initial value y0: 1
Enter step size h: 0.1
Enter the point xp: 0.2

Step      x          y
1         0.1000    1.1100
2         0.2000    1.2421

The value of y at x = 0.2000 is: 1.242050
>> |
```